



Gruppo SkyNet
skynetunigroup@gmail.com

Progettazione Infrastruttura AWS (MVP)

Versione	<i>1.0</i>
Uso	<i>Interno</i>
Redattori	<i>Dario Pirrone</i>
Verifica	<i>Nome Cognome</i>
Approvazione	<i>Nome Cognome</i>

Indice

Indice	2
Elenco delle tabelle	3
Elenco delle figure	4
I Manuale Operativo	5
1 Guida Operativa di Rilascio (Checklist)	5
1.1 Attività da avviare il Giorno 1	5
1.2 Rete e Sicurezza	5
1.3 Segreti e Chiavi	6
1.4 Dati e Caching	7
1.5 Immagini e Registro (ECR)	8
1.6 Calcolo e Orchestrazione (ECS)	8
1.7 Frontend e Distribuzione	8
1.8 Storage Applicativo	9
1.9 Osservabilità e Budget	9
2 Perimetro e Scope	10
2.1 Servizi AWS per Area (MVP)	10
2.2 Fuori Perimetro	10
2.3 Mappatura Componenti PoC → AWS	11
3 Rete e Sicurezza (VPC)	11
3.1 Parametri VPC base	11
3.2 Subnet e Routing	12
3.3 VPC Endpoints (PrivateLink)	12
3.4 Matrice Security Groups (Regole di Rete)	13
4 Compute e Orchestrazione (ECS su Fargate)	14
4.1 Cluster ECS e Task Definition	14
4.2 Gestione Immagini Container (Amazon ECR)	15
4.3 Application Load Balancer (ALB)	15
4.4 Service Discovery (AWS Cloud Map)	15
4.5 Flussi Applicativi End-to-End	16
5 Livello Dati e Caching	16
5.1 MongoDB Atlas (via AWS PrivateLink)	16
5.2 Amazon ElastiCache (Redis)	17
6 Integrazione Amazon Bedrock	18
6.1 Modelli e Accessi	18
6.2 Autenticazione e Policy IAM	19
6.3 Model Invocation Logging	19
Progettazione Infrastruttura AWS (MVP)	2

7	Storage e Comunicazioni Esterne	19
7.1	Amazon S3 (Artefatti Applicativi)	19
8	Frontend Web Hosting	19
8.1	Amazon S3 (Hosting Statico)	20
8.2	Distribuzione Amazon CloudFront	20
8.3	Gestione SPA Routing e Build Vite	20
9	Segreti e Configurazione (Variabili d'Ambiente)	20
9.1	Mappatura Variabili PoC → AWS	20
9.2	Cifratura (AWS KMS)	21
10	Budget e Osservabilità Infrastrutturale	21
10.1	Metriche e Allarmi Base	21
10.2	Gestione Costi (AWS Budgets)	22
11	Tracciamento Requisito → Decisione AWS	22
II	Motivazioni e Decisioni Architettureali	24
12	Principi Guida e Vincoli di Progetto	25
13	Decisioni Architettureali (ADR) e Motivazioni	25
13.1	Identità, Accessi e Segreti	25
13.2	Rete, Sicurezza e Isolamento	26
13.3	Compute, Orchestrazione e Frontend	27
13.4	Scelte sul Livello Dati	27
13.5	Scelta dei Modelli LLM (Bedrock)	27
14	Compromessi e Rischi Accettati	28
15	Limiti di Piattaforma (Fargate e Prompt)	30

Elenco delle tabelle

1	Mappa dei servizi AWS dell'MVP.	10
2	Mappatura operativa PoC verso AWS.	11
3	Parametri di Subnetting.	12
4	Route table della VPC e relative associazioni.	12
5	Elenco VPC Endpoints da configurare.	13
6	Matrice delle Regole Security Group. *L'endpoint Bedrock accetta ingress solo da sg-agents	14
7	Dimensionamento e configurazione Task ECS per l'MVP.	14
8	Configurazione dei registri ECR.	15

9	Configurazione ALB.	15
10	Record Service Discovery Cloud Map.	15
11	Configurazione MongoDB Atlas.	17
12	Configurazione ElastiCache (Redis).	17
13	Assegnazione Modelli e Richieste di Accesso Bedrock.	18
14	Configurazione IAM per l'accesso a Bedrock.	19
15	Configurazione Bucket S3 (Artefatti).	19
16	Behavior di CloudFront per l'MVP condiviso.	20
17	Mappatura delle Variabili d'Ambiente.	21
18	Matrice degli allarmi infrastrutturali base.	22
19	Mappatura Requisiti - Decisioni Infrastrutturali AWS.	24
20	Matrice dei Compromessi e Rischi Accettati per l'MVP.	30

Elenco delle figure

1	VPC del MVP: due Availability Zone, subnet pubbliche e private, NAT Gateway unico in public-a . Il database MongoDB Atlas è esterno alla VPC e raggiunto via AWS PrivateLink (Interface Endpoint "Atlas PL" nelle subnet private). Gli altri VPC Endpoint verso i servizi AWS interni sono omessi da questo diagramma.	12
2	Flussi applicativi principali con protocollo e direzione. Il traffico agents→GitHub non esiste: ogni lettura passa dalla facade del backend (§3, ADR-AWS-1).	16

Parte I

Manuale Operativo

Questa sezione contiene esclusivamente i parametri di configurazione, le tabelle di mappatura e le istruzioni operative per il rilascio. Tutte le argomentazioni, le motivazioni delle scelte e le assunzioni di rischio sono documentate nella Parte II.

1 Guida Operativa di Rilascio (Checklist)

Nota Implementativa

Questa sequenza descrive le **dipendenze logiche** tra risorse, non l'ordine di scrittura del codice CDK. CDK/CloudFormation crea in parallelo tutto ciò che non ha dipendenze esplicite; se una risorsa referencia l'oggetto/ARN di un'altra, la dipendenza è inferita automaticamente, altrimenti va dichiarata con `.node.addDependency()`.

Questa sequenza riflette le dipendenze tecniche tra le risorse. Le fasi indipendenti (es. richiesta Bedrock e Rete) possono procedere in parallelo.

1.1 Attività da avviare il Giorno 1

Queste richieste hanno tempi di approvazione AWS non controllabili dal team:

- **Amazon Bedrock:** Richiedere l'accesso ai modelli nella regione `eu-south-1`. *Verificare in anticipo l'assenza di Service Control Policy (SCP) organizzative bloccanti.*

1.2 Rete e Sicurezza

1. Creazione VPC (10.0.0.0/16) su due AZ, con `enableDnsSupport` e `enableDnsHostnames` attivi.
2. Creazione 4 subnet (2 pubbliche, 2 private), 2 route table e Internet Gateway.
3. Creazione NAT Gateway singolo in `public-a`.

Attenzione

Il traffico da `private-b` verso il NAT Gateway in `public-a` attraversa un confine di Availability Zone: AWS addebita i costi di **trasferimento dati cross-AZ** in aggiunta al costo orario/per-GB del NAT stesso. Non è solo un rischio di disponibilità (Parte II) ma una voce di costo da includere nella stima con AWS Pricing Calculator (§10.2).

4. Creazione VPC Endpoints con Private DNS abilitato.

Nota Implementativa

Poiché i task ECS girano in subnet private senza IP pubblico, il pull delle immagini da ECR avviene esclusivamente tramite i VPC Endpoint `api/dkr` configurati in questo stesso punto: non è necessaria alcuna route verso Internet per il deploy dei container.

5. Creazione Security Group rispettando l'ordine di dipendenza: prima `sg-alb`, `sg-atlas`, `sg-redis`, `sg-vpce`; successivamente `sg-backend` e `sg-agents`.

Attenzione

Le regole tra `sg-backend` e `sg-agents` sono **reciproche** (backend → agents in egress, agents → backend in ingress, e viceversa). Definirle inline nel costruttore del `SecurityGroup` genera un errore di **dipendenza circolare** in CloudFormation. Istanziare i due SG vuoti e aggiungere le regole *dopo* con `addIngressRule()` / `addEgressRule()`:

Listing 1: Pattern corretto per evitare dipendenza circolare tra SG

```

1 // 1. Istanziare i SG vuoti
2 const sgBackend = new ec2.SecurityGroup(this, 'SgBackend', { vpc });
3 const sgAgents = new ec2.SecurityGroup(this, 'SgAgents', { vpc });
4
5 // 2. Aggiungere le regole reciproche DOPO la creazione
6 sgBackend.addEgressRule(sgAgents, ec2.Port.tcp(8000), 'To agents');
7 sgAgents.addIngressRule(sgBackend, ec2.Port.tcp(8000), 'From backend');
8
9 sgAgents.addEgressRule(sgBackend, ec2.Port.tcp(3000), 'To backend');
10 sgBackend.addIngressRule(sgAgents, ec2.Port.tcp(3000), 'From agents');
```

1.3 Segreti e Chiavi

1. Creazione chiave KMS condivisa (Parte II: Compromessi e Rischi).

Attenzione

La cifratura a riposo di **ElastiCache** deve essere specificata *al momento della creazione*: non è possibile abilitarla in un secondo momento su un nodo esistente. Se dimenticata, l'unica soluzione è ricreare la risorsa da zero. Verificare la key policy della CMK: deve concedere esplicitamente l'uso ai Task Execution Role di `backend` e `agents`, altrimenti anche con permessi IAM corretti le chiamate falliscono con `KMS.AccessDeniedException`.

2. Creazione voci in Secrets Manager e Parameter Store con valori effettivi (vietati i placeholder in ambiente condiviso).

Attenzione

Se un segreto in Secrets Manager è salvato come oggetto JSON, l'ARN referenziato nella Task Definition deve specificare la chiave da estrarre con la sintassi `<secret-arn>:<jsonKey>::`, altrimenti ECS inietta l'intero blob JSON come valore della variabile d'ambiente:

Listing 2: Estrazione di una singola chiave da un secret JSON

```

1 containerDefinitions: [{
2   secrets: [{
3     name: 'MONGO_URI',
4     valueFrom: '${secret.secretArn}:MONGO_URI:', // nota il suffisso
5   }],
6 }]

```

3. Creazione ruoli IAM separati: Task Execution Role e Task Role per backend e agents.

Nota Implementativa

Chi arriva da Docker Compose spesso confonde i due ruoli IAM di ECS. La tabella seguente chiarisce la differenza:

Ruolo	Usato da	Permessi tipici
Task Execution Role	L'agente ECS, <i>prima</i> che il container parta	Pull immagine da ECR, lettura segreti da iniettare come env var, scrittura log su Cloud-Watch
Task Role	Il codice applicativo, <i>dentro</i> il container in esecuzione	Chiamate API AWS runtime: bedrock:InvokeModel , accesso S3, ecc.

Attenzione

Assegnare i permessi Bedrock/S3 al ruolo sbagliato (es. Execution Role invece di Task Role) non impedisce l'avvio del container: la chiamata AWS fallisce a runtime con **AccessDenied**, un errore silenzioso e difficile da diagnosticare se non si sa dove guardare.

1.4 Dati e Caching

1. Provisioning cluster **MongoDB Atlas** (tier M10, regione AWS eu-south-1) tramite console/API Atlas o Atlas Terraform/CDK provider.
2. Configurazione **AWS PrivateLink** verso Atlas: creazione del Private Endpoint dal progetto Atlas e dell'Interface VPC Endpoint corrispondente lato AWS (endpoint service fornito da Atlas) su **private-a/private-b**.
3. Creazione cluster ElastiCache Redis (nodo singolo), subnet group privato e cifratura KMS.
4. *Azione Dev*: Recuperare da Atlas la *connection string del Private Endpoint*, restringere la *Network Access List* al solo Private Endpoint e verificare la connettività dal task **backend**. Nessuno spike di compatibilità richiesto: Atlas è MongoDB nativo.

Nota Implementativa

A differenza di DocumentDB, Atlas è MongoDB nativo: non esistono divergenze di compatibilità sul driver Mongoose (operatori di aggregazione, transazioni multi-documento, tipi di indice e upsert su array nidificati si comportano come in un MongoDB self-hosted). L'attività di setup si sposta quindi dalla validazione applicativa alla **configurazione della connettività PrivateLink** e della *Network Access List* lato Atlas.

1.5 Immagini e Registro (ECR)

1. Creazione repository ECR codeguardian/backend e codeguardian/agents.
2. Attivazione scansione vulnerabilità e Lifecycle Policy.
3. Creazione ruolo IAM CI/CD per push immagini.
4. Primo push manuale per test del flusso.

1.6 Calcolo e Orchestrazione (ECS)

1. Creazione cluster ECS (Capacity provider: FARGATE).
2. Creazione Task Definition (backend, agents) con iniezione segreti via ARN.
3. Creazione namespace Cloud Map (codeguardian.local).
4. Creazione servizi ECS (Desired count: 1, Rete: awsvpc, Service Discovery abilitata).

Attenzione

Se il container **backend** impiega alcuni secondi ad avviarsi (connessione a MongoDB Atlas/Redis, caricamento moduli NestJS), l'ALB può marcarlo **unhealthy** e forzarne il riavvio prima ancora che sia pronto, generando un **restart-loop**. Impostare esplicitamente **healthCheckGracePeriod** sul servizio ECS:

Listing 3: Grace period per evitare restart-loop in avvio

```
1 const service = new ecs.FargateService(this, 'BackendService', {
2   cluster,
3   taskDefinition,
4   desiredCount: 1,
5   healthCheckGracePeriod: cdk.Duration.seconds(60),
6 });
```

5. Creazione Application Load Balancer (ALB), target group (ip su porta 3000) e listener (HTTP 80).

1.7 Frontend e Distribuzione

1. Creazione bucket S3 statico (privato, Origin Access Control).

Attenzione

"Origin Access Control" da solo **non basta**: va aggiunta esplicitamente una *bucket policy* che conceda `s3:GetObject` al principal `cloudfront.amazonaws.com`, con una condizione su `AWS:SourceArn` che punta alla distribuzione. Dimenticarlo è la causa più comune di 403 Forbidden su tutto il sito dopo il deploy.

Listing 4: Bucket policy minima per accesso via OAC

```

1 {
2   "Effect": "Allow",
3   "Principal": { "Service": "cloudfront.amazonaws.com" },
4   "Action": "s3:GetObject",
5   "Resource": "arn:aws:s3:::<bucket-name>/*",
6   "Condition": {
7     "StringEquals": {
8       "AWS:SourceArn": "arn:aws:cloudfront::<account-id>:distribution/<dist-id>"
9     }
10  }
11 }
```

2. Creazione distribuzione CloudFront condivisa (Behavior /* verso S3, Behavior /api/* e /socket.io/* verso ALB).

Nota Implementativa

Una distribuzione CloudFront impiega tipicamente **15–20 minuti** per la propagazione iniziale completa (pochi minuti per le modifiche successive). Non vedere il sito funzionante subito dopo il primo deploy è comportamento atteso, non un errore di configurazione.

3. Configurazione Custom Error Response (403/404 → `index.html`).
4. Build di Vite con `VITE_API_BASE_URL` relativo, caricamento e invalidazione cache.

1.8 Storage Applicativo

1. Creazione bucket S3 per export report (privato, policy limitata al Task Role del backend, Lifecycle 30 giorni).

1.9 Osservabilità e Budget

1. Creazione Log Group CloudWatch e resource policy per Bedrock invocation logging.

Attenzione

Un CloudWatch Log Group ha di default **retention infinita** (*Never Expire*), con costo di storage che cresce indefinitamente. Impostare una retention esplicita su ogni Log Group creato:

Listing 5: Retention esplicita sul Log Group

```

1 const logGroup = new logs.LogGroup(this, 'BackendLogs', {
2   retention: logs.RetentionDays.TWO_WEEKS,
3 });
```

2. Abilitazione Container Insights su ECS.
3. Creazione Argomento SNS (`codeguardian-alerts`) e sottoscrizione email del team.

Attenzione

La sottoscrizione email a SNS richiede un **click di conferma** nella mail ricevuta. Se nessuno lo effettua, gli allarmi non verranno mai recapitati e il problema resta invisibile finché non serve davvero. Verificare lo stato **Confirmed** della subscription in console SNS prima di considerare questo punto completato.

4. Creazione allarmi operativi CloudWatch verso SNS.
5. Calcolo stima tramite AWS Pricing Calculator e creazione allarmi AWS Budgets verso SNS.

2 Perimetro e Scope

2.1 Servizi AWS per Area (MVP)

Area	Servizi AWS Configurati
Rete e Sicurezza	VPC dedicata (2 AZ), Subnet pubbliche/private, 1 NAT Gateway, VPC Endpoints, Security Group dedicati.
Compute	Amazon ECS su Fargate, Amazon ECR, ALB, AWS Cloud Map (Service Discovery).
Dati e Caching	MongoDB Atlas (cluster M10, via AWS PrivateLink), Amazon ElastiCache Redis (1 nodo).
LLM	Amazon Bedrock (Famiglia Qwen3 in eu-south-1).
Storage e Frontend	Amazon S3 (2 bucket: artefatti e build React), Amazon CloudFront (distribuzione condivisa).
Segreti	AWS Secrets Manager, Parameter Store, 1 Chiave AWS KMS.
Osservabilità & Costi	Amazon CloudWatch Logs/Insights, AWS Budgets, SNS.

Tabella 1: Mappa dei servizi AWS dell'MVP.

2.2 Fuori Perimetro

Le seguenti architetture **non** fanno parte dell'MVP (Parte II: Compromessi e Rischi Accettati):

- Auto-scaling dinamico per ECS.
- Alta disponibilità avanzata / Multi-Region (Single NAT e nodo Redis singolo accettati; il cluster Atlas M10 è già multi-AZ nella regione).
- Dominio Custom (Route53) e relativi certificati validati.

- AWS WAF (Web Application Firewall).

2.3 Mappatura Componenti PoC → AWS

L'unità di deploy (immagine Docker) e il codice applicativo **non subiscono modifiche** infrastrutturali (Parte II: Principio di Migrazione).

Componente PoC	Target AWS	Azione / Vincolo per i Dev
Container backend e agents	ECS Fargate	Sostituire variabili d'ambiente. lo: I prompt non possono usare mount; richiedono bake-in e dell'immagine per le modifiche.
Frontend Vite (Dev Server)	S3 + CloudFront	Modificare configurazione Vite. Pilazione statica con URL base. Nessuna gestione CORS ne (Parte II: ADR-AWS-9).
Rete Docker (agents:8000)	Cloud Map	URL aggiornato <code>agents.codeguardian.local</code> .
LLM API Esterna (DashScope)	Amazon Bedrock	Passaggio da chiave API statica a messi IAM (Task Role).
MongoDB	MongoDB Atlas (via PrivateLink)	Aggiornare <code>MONGO_URI</code> con la <i>connection string del Private Endp</i> las. Nessuna validazione di con ità necessaria (MongoDB nativ
Redis	ElastiCache	Aggiornare <code>REDIS_URL</code> . Dupl (coda + cache) mantenuto sul istanza.
MinIO	Amazon S3	Aggiornare l'endpoint SDK n end. Passaggio ad autent tramite Task Role.

Tabella 2: Mappatura operativa PoC verso AWS.

3 Rete e Sicurezza (VPC)

3.1 Parametri VPC base

- **Strumento IaC:** AWS CDK (TypeScript).
- **Regione:** eu-south-1 (Milano).
- **CIDR VPC:** 10.0.0.0/16.
- **Availability Zones:** eu-south-1a, eu-south-1b.
- **Impostazioni DNS:** enableDnsSupport=true, enableDnsHostnames=true.

3.2 Subnet e Routing

Subnet	CIDR	Tipo	Risorse allocate
public-a	10.0.0.0/24	Pubblica	ALB, NAT Gateway (Parte II: Rischio Single-NAT)
private-a	10.0.1.0/24	Privata	ECS, ElastiCache, VPC Endpoints (incl. Atlas PrivateLink)
public-b	10.0.2.0/24	Pubblica	ALB (nodo AZ secondaria)
private-b	10.0.3.0/24	Privata	ECS, ElastiCache, VPC Endpoints (incl. Atlas PrivateLink)

Tabella 3: Parametri di Subnetting.

Route table	Associazione	Regola
rt-public	public-a, public-b	0.0.0.0/0 → Internet Gateway
rt-private	private-a, private-b	0.0.0.0/0 → NAT Gateway (public-a)

Tabella 4: Route table della VPC e relative associazioni.

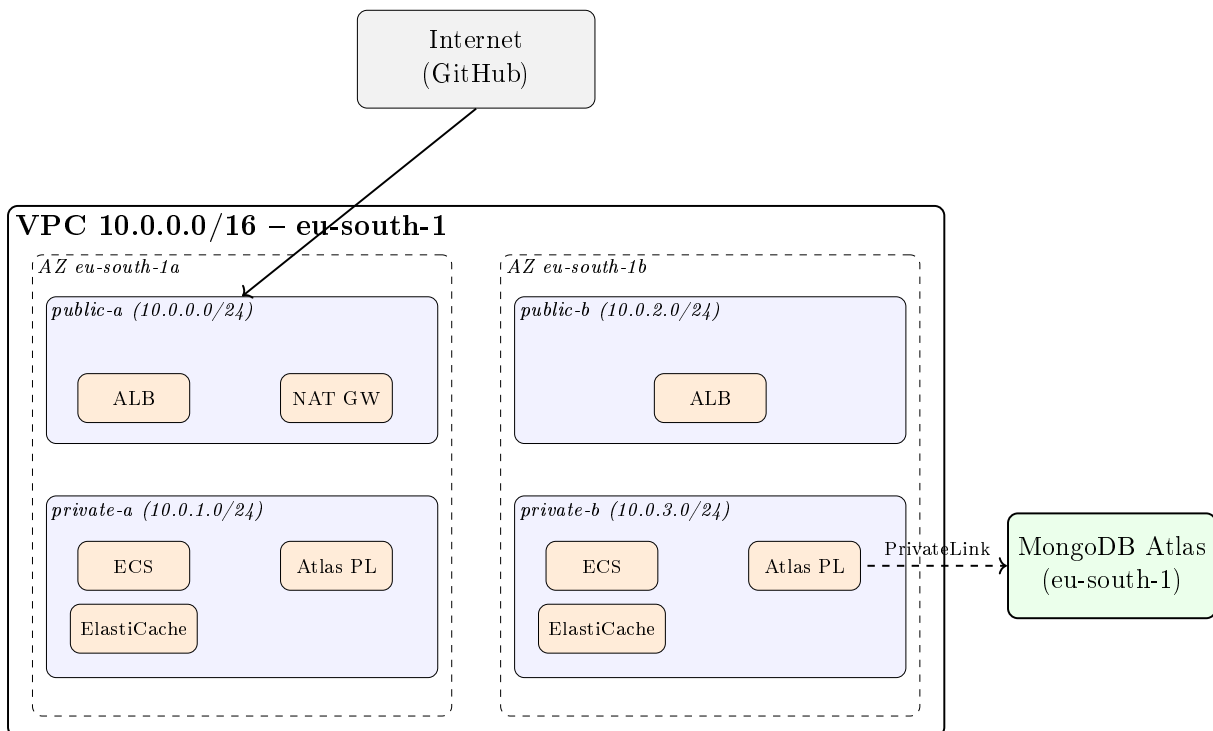


Figura 1: VPC del MVP: due Availability Zone, subnet pubbliche e private, NAT Gateway unico in **public-a**. Il database MongoDB Atlas è esterno alla VPC e raggiunto via AWS PrivateLink (Interface Endpoint "Atlas PL" nelle subnet private). Gli altri VPC Endpoint verso i servizi AWS interni sono omessi da questo diagramma.

3.3 VPC Endpoints (PrivateLink)

Tutto il traffico AWS interno deve restare nella VPC. Il NAT serve esclusivamente per GitHub. Tutti gli Endpoint "Interface" richiedono l'abilitazione del **Private DNS**.

Servizio AWS	Tipo	Note Operative
S3	Gateway	Agisce su Route Table, no Private DNS necessario.
Bedrock Runtime	Interface	Indispensabile per sicurezza Prompt (Parte II: ADR-AWS-6).
Secrets Manager	Interface	Interrogato dai task all'avvio.
ECR (api, dkr)	Interface	Pull immagini senza uscita web.
CloudWatch Logs	Interface	Traffico streaming logs continuo.
SSM, SSM Msgs, EC2 Msgs	Interface	Necessari per ECS Exec senza Bastion Host (Parte II: ADR-AWS-7).
MongoDB Atlas (PrivateLink)	Interface	Endpoint verso l' <i>endpoint service</i> di Atlas (non <code>com.amazonaws.*</code>). Usare la <i>connection string del Private Endpoint</i> fornita da Atlas per la risoluzione DNS.

Tabella 5: Elenco VPC Endpoints da configurare.

Nota Implementativa

Ogni VPC Endpoint di tipo *Interface* viene fatturato per ora **per ogni AZ** in cui è distribuito. Con 7 endpoint Interface (incluso Atlas PrivateLink) su 2 AZ si tratta di 14 ENI fatturati continuativamente, da includere nella stima con AWS Pricing Calculator (§10.2) insieme al NAT Gateway.

3.4 Matrice Security Groups (Regole di Rete)

Ogni componente possiede un SG dedicato. Il traffico non in elenco è implicitamente negato.

Security Group	Ingress (In Entrata)	Egress (In Uscita)
sg-alb	TCP 80 dal Prefix List AWS <code>com.amazonaws.global.cloudfront.origin-facing</code>	TCP 3000 verso sg-backend
sg-backend	TCP 3000 da sg-alb	TCP 27017 verso sg-atlas (endpoint Atlas PrivateLink) TCP 6379 verso sg-redis TCP 443 verso sg-vpce Porta interna verso sg-agents TCP 443 verso 0.0.0.0/0 (Solo NAT/GitHub)
sg-agents	Porta interna da sg-backend	Porta interna verso sg-backend TCP 443 verso sg-vpce NESSUNA regola verso 0.0.0.0/0 (Parte II: ADR-AWS-4)

Security Group	Ingress (In Entrata)	Egress (In Uscita)
sg-atlas	TCP 27017 da sg-backend	(Nessuna regola)
sg-redis	TCP 6379 da sg-backend	(Nessuna regola)
sg-vpce	TCP 443 da sg-backend TCP 443 da sg-agents*	(Nessuna regola)

Tabella 6: Matrice delle Regole Security Group. *L'endpoint Bedrock accetta ingress **solo** da sg-agents.

Attenzione

Il Security Group `sg-atlas` è associato all'*Interface VPC Endpoint* di Atlas PrivateLink, non a un'istanza database nella VPC. La porta esatta (27017 o l'eventuale range indicato da Atlas) va confermata dalla *connection string del Private Endpoint* generata in fase di pairing e usata come sorgente di verità per le regole di egress di sg-backend.

4 Compute e Orchestrazione (ECS su Fargate)

Il deploy dell'applicativo si basa sulle stesse immagini Docker validate nel PoC, senza modifiche al codice sorgente (Parte II: Principio di Migrazione).

4.1 Cluster ECS e Task Definition

- **Capacity Provider:** Esclusivamente `FARGATE` (nessun `FARGATE_SPOT`).
- **Modalità di Rete:** `awsvpc` (interfacce dedicate nelle subnet private).
- **Log Driver:** `awslogs` (inoltre standard output/error a CloudWatch Logs).

Servizio ECS	vCPU	Memoria	Desired Count	Configurazioni Task
backend	0.5	1 GB	1 (Parte II)	Health check breve su <code>/health</code> . Vincolo implementativo: la generazione PDF deve usare una libreria di composizione programmatica (es. <code>pdftk</code> / <code>pdf-lib</code>), non rendering via browser headless; dimensionamento da confermare con test di carico su export concorrenti prima del rilascio (Parte II, Tabella 20).
agents	1.0	2 GB	1 (Parte II)	Valori di base, aggiornabili dopo test di carico su LangGraph e Bedrock.

Tabella 7: Dimensionamento e configurazione Task ECS per l'MVP.

4.2 Gestione Immagini Container (Amazon ECR)

Parametro	Valore / Configurazione
Repository	codeguardian/backend, codeguardian/agents
Tagging	Uso obbligatorio dello SHA breve del commit (es. :a1b2c3d). Tag <code>latest</code> solo come puntatore di comodo in dev.
Security Scan	Scansione automatica base abilitata al push.
Lifecycle Policy	Mantenere ultime 20 immagini taggate; eliminare immagini untagged vecchie di > 1 giorno.
Permessi IAM Push	Ruolo CI/CD (<code>codeguardian-ci-role</code>).
Permessi IAM Pull	Task Execution Role di ECS.

Tabella 8: Configurazione dei registri ECR.

4.3 Application Load Balancer (ALB)

Nessun dominio custom o certificato TLS è installato sull'ALB (Parte II: ADR-AWS-8).

Parametro	Valore Operativo
Posizionamento	Subnet pubbliche (<code>public-a</code> , <code>public-b</code>).
Listener	Protocollo HTTP, Porta 80.
Target Group	Tipo <code>ip</code> , Porta 3000, Protocollo HTTP (puntato ai task del backend). Path Health Check: <code>/health</code> .
Idle Timeout	3600 secondi (Necessario per connessioni WebSocket).
Stickiness	Abilitata, cookie <code>AWSALB</code> , durata 3600 secondi .
Deregistration	30 secondi (Ridotto per deploy più rapidi).

Tabella 9: Configurazione ALB.

4.4 Service Discovery (AWS Cloud Map)

Usata per la comunicazione interna (Backend ↔ Agents).

- **Namespace Privato:** `codeguardian.local` (associato alla VPC).
- **TTL Record DNS:** 10 secondi.

Servizio DNS	Porta	Tipo	Scopo
<code>agents.codeguardian.local</code>	8000	A	Chiamato dal backend per avvio run.
<code>backend.codeguardian.local</code>	3000	A	Chiamato dagli agenti per tool-calls (GitHub read-only) e callback di progresso (Parte II: ADR-AWS-3).

Tabella 10: Record Service Discovery Cloud Map.

4.5 Flussi Applicativi End-to-End

Il diagramma di rete (Figura 1) mostra il posizionamento delle risorse, non i flussi dati. La Figura 2 integra le due viste, mostrando chi chiama chi e con quale protocollo.

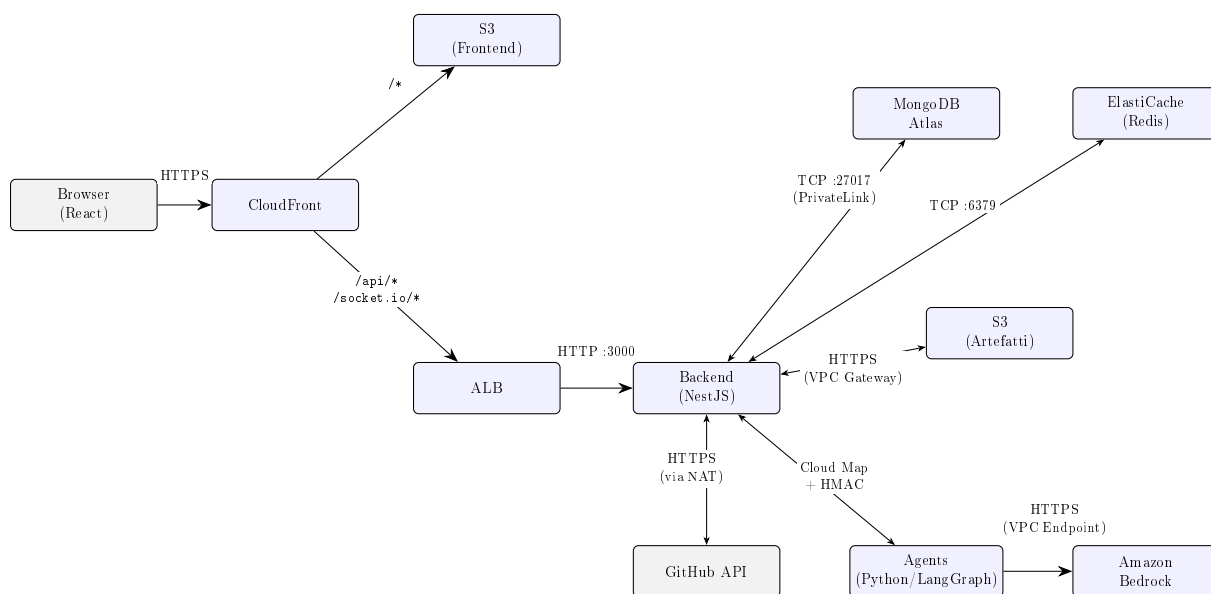


Figura 2: Flussi applicativi principali con protocollo e direzione. Il traffico agents→GitHub non esiste: ogni lettura passa dalla facade del backend (§3, ADR-AWS-1).

5 Livello Dati e Caching

5.1 MongoDB Atlas (via AWS PrivateLink)

Sostituisce il container MongoDB del PoC con un cluster **MongoDB Atlas** gestito, deployato su AWS nella regione **eu-south-1** e raggiunto esclusivamente tramite **AWS PrivateLink** (nessun transito su Internet pubblico e nessuna dipendenza dal NAT Gateway per il livello dati).

Parametro	Valore Operativo
Tier	M10 dedicato (minimo che abilita PrivateLink).
Topologia	Replica set a 3 nodi distribuiti sulle AZ della regione (alta disponibilità per costruzione, gestita da Atlas).
Regione	AWS eu-south-1 (Milano), per data residency coerente col resto dello stack.
Versione	Ultima major MongoDB supportata da Atlas nella regione.
Connettività	AWS PrivateLink (Interface Endpoint su <code>private-a/private-b</code>); <i>Network Access List</i> Atlas ristretta al Private Endpoint.
Rete e Sicurezza	Porta MongoDB (27017), SG <code>sg-atlas</code> sull'endpoint (Ingresso solo da <code>sg-backend</code>).

Parametro	Valore Operativo
Cifratura	A riposo gestita da Atlas (AES-256); BYOK via KMS fuori scope MVP. In transito TLS obbligatoria (CA pubblica).
Backup	Atlas Cloud Backup (snapshot gestiti da Atlas, non da AWS Backup).

Tabella 11: Configurazione MongoDB Atlas.

Attenzione

Con PrivateLink la connessione deve usare la *connection string del Private Endpoint* generata da Atlas (host che risolvono a IP privati), **non** la connection string standard `mongodb+srv://` pubblica: quest'ultima risolverebbe a IP pubblici non instradabili senza NAT, con connessione in timeout senza un errore esplicito. Atlas usa certificati TLS da CA pubblica: **non serve alcun CA bundle custom** (a differenza di DocumentDB, che imponeva il bundle RDS).

Listing 6: Connessione Mongoose a MongoDB Atlas (Private Endpoint)

```

1 // MONGO_URI = connection string del PRIVATE ENDPOINT
2 // (Atlas -> Connect -> Private Endpoint), es.:
3 // mongodb+srv://<user>:<pwd>@<cluster>-pl-0.<hash>.mongodb.net/<db>?retryWrites=true
4
5 mongoose.connect(process.env.MONGO_URI, {
6   // TLS implicito con mongodb+srv; nessun tlsCAFile custom (CA pubblica)
7 });

```

Azione Dev: Configurare Private Endpoint e Network Access List su Atlas e verificare la connessione dal task `backend`. Lo schema design della base dati resta di competenza del team backend (RV.21), senza spike di compatibilità.

5.2 Amazon ElastiCache (Redis)

Duplica uso applicativo: coda BullMQ (`bull:*`) e cache GitHub (`repo@sha:*`).

Parametro	Valore Operativo
Topologia	Nodo singolo (No cluster, no repliche) (Parte II).
Subnet Group	Subnet private (<code>private-a</code> , <code>private-b</code>).
Rete e Sicurezza	Porta 6379, SG <code>sg-redis</code> (Ingresso solo da <code>sg-backend</code>).
Cifratura	A riposo abilitata (Stessa chiave KMS), in transito (<code>in-transit encryption</code>) abilitata.

Tabella 12: Configurazione ElastiCache (Redis).

Attenzione

Abilitare `in-transit encryption` su ElastiCache **non è trasparente** lato applicativo: il client (`ioredis` / `node-redis`) deve connettersi specificando esplicitamente l'opzione TLS, altrimenti la connessione fallisce in timeout senza un messaggio di errore chiaro.

Listing 7: Connessione ioredis con TLS abilitato

```

1 const redis = new Redis({
2   host: process.env.REDIS_HOST,
3   port: 6379,
4   tls: {}, // richiesto se in-transit encryption e' abilitata
5 });

```

Rate Limiting Applicativo (RQ.8)

Oltre agli usi già previsti (coda BullMQ, cache letture GitHub), l'istanza Redis supporta un **rate limiter applicativo** lato backend NestJS (token bucket o sliding window, chiave per utente/IP), che limita attivamente il volume di richieste verso gli endpoint che invocano Bedrock o l'API GitHub. Questo copre RQ.8 con un controllo architetturale reale, senza affidarsi ai soli limiti di default dell'account Bedrock (insufficienti da soli a soddisfare il requisito in modo verificabile). Non introduce nuovi servizi AWS: riusa l'istanza ElastiCache già provisionata.

6 Integrazione Amazon Bedrock

Tutte le comunicazioni con Bedrock avvengono via VPC Endpoint, garantendo che il codice analizzato non transiti su Internet pubblico (Parte II: Conformità RS.5).

6.1 Modelli e Accessi

L'accesso ai modelli va richiesto **immediatamente** tramite console Bedrock (sezione *Model access*) nella regione eu-south-1.

Agente	Modello Selezionato	Priorità di Richiesta
Docs, Changelog	Qwen3-32B (dense)	Alta (Assegnazione primaria).
Security	Qwen3-Coder-30B-A3B-Instruct	Alta (Assegnazione primaria).
N/A (Scalata)	Qwen3-235B-A22B	Media (Richiedere subito come fallback in caso di accuratezza insufficiente in fase di validazione) (Parte II).

Tabella 13: Assegnazione Modelli e Richieste di Accesso Bedrock.

Attenzione

Verificare in console Bedrock (sezione *Model access* / *Cross-region inference*) se i modelli Qwen3 richiedono un **Inference Profile ARN** invece dell'ARN diretto del foundation model per l'invocazione on-demand. Se richiesto, l'ARN nella policy IAM e nel codice applicativo va aggiornato di conseguenza; l'errore tipico in caso di mismatch è `ValidationException: Invocation of model ID ... is not supported`.

6.2 Autenticazione e Policy IAM

Nessuna chiave API statica (Parte II: ADR-AWS-1). L'invocazione avviene tramite il **Task Role** del servizio **agents**.

Parametro Policy	Valore Operativo
Azioni Ammesse	<code>bedrock:InvokeModel, bedrock:InvokeModelWithResponseStream</code>
Risorse (ARNs)	<code>arn:aws:bedrock:eu-south-1::foundation-model/qwen.*</code> (Scoping limitato alla famiglia Qwen3 per impedire l'uso accidentale di modelli costosi).
Condizioni	<code>aws:RequestedRegion == eu-south-1.</code>

Tabella 14: Configurazione IAM per l'accesso a Bedrock.

6.3 Model Invocation Logging

- **Cosa registrare: Solo i metadati** (Modello invocato, timestamp, token consumati).
- **Cosa NON registrare:** Corpo del prompt e della risposta (per policy di sicurezza sul codice).
- **Vincolo:** Aggiungere resource policy al Log Group per permettere al principal `bedrock.amazonaws.com` la scrittura.

7 Storage e Comunicazioni Esterne

7.1 Amazon S3 (Artefatti Applicativi)

Sostituisce MinIO per il salvataggio dei report generati.

Parametro	Valore Operativo
Accesso di Rete	Nessun accesso pubblico (<code>Block Public Access = True</code>). Accesso solo via VPC Endpoint Gateway.
Permessi IAM	Concessi in lettura/scrittura esclusivamente al Task Role del servizio backend .
Cifratura & Versioning	Cifratura KMS abilitata. Versioning disabilitato.
Lifecycle Policy	Eliminazione automatica oggetti > 30 giorni.

Tabella 15: Configurazione Bucket S3 (Artefatti).

8 Frontend Web Hosting

Il frontend React (Vite) viene compilato come sito statico. Un'unica distribuzione CloudFront serve sia il Frontend sia le API, eliminando la necessità di configurare policy CORS (Parte II: ADR-AWS-9).

8.1 Amazon S3 (Hosting Statico)

- **Accesso Pubblico:** Bloccato. Il Bucket NON deve usare la funzione "Static Website Hosting" nativa.
- **Accesso in Lettura:** Esclusivo per CloudFront tramite Origin Access Control (OAC).

8.2 Distribuzione Amazon CloudFront

Behavior Path	Origin Target	Configurazione e Policy
/* (Default)	Bucket S3	Viewer Protocol: Redirect HTTP to HTTPS. Cache: Abilitata.
/api/* e /socket.io/*	ALB (Backend)	Viewer Protocol: HTTP Only (L'ALB non ha certificato TLS). Cache: Disabilitata. Origin Request Policy: Forward All (Inoltro header Upgrade, Connection, e Authorization).

Tabella 16: Behavior di CloudFront per l'MVP condiviso.

8.3 Gestione SPA Routing e Build Vite

- **Custom Error Response (CloudFront):** Errori 403/404 generati da S3 devono essere redirezionati al file `index.html` con Status Code 200 (necessario per TanStack Router).
- **Variabili Runtime:** Configurare `VITE_API_BASE_URL` con un path relativo (es. `/api/v1`) durante la build.
- **Cache Invalidation:** Il ruolo CI/CD deve avere i permessi `cloudfront:CreateInvalidation` per svuotare la cache ad ogni deploy.

9 Segreti e Configurazione (Variabili d'Ambiente)

9.1 Mappatura Variabili PoC → AWS

Integrazione trasparente: il codice applicativo continua a leggere le variabili tramite `process.env` / `os.environ`. La configurazione viene iniettata nativamente da ECS al momento del bootstrap.

Variabile (PoC)	Servizio	Meccanismo AWS	Permessi IAM	Lettura
MONGO_URI	backend	Secrets Manager	Task (backend)	Execution

Variabile (PoC)	Servizio	Meccanismo AWS	Permessi IAM	Lettura
JWT_SECRET	backend	Secrets Manager	Task (backend)	Execution
CREDENTIAL_MASTER_KEY	backend	Secrets Manager	Task (backend)	Execution
INTERNAL_SHARED_SECRET	Entrambi	Secrets Manager	Task Exec (backend & agents)	
REDIS_URL	backend	Parameter Store	Task (backend)	Execution
BACKEND_BASE_URL	agents	Parameter Store	Task (agents)	Execution
LLM_PROVIDER	agents	Parameter Store	Task (agents)	Execution
<i>Variabili Rimosse o Modificate</i>				
LLM_API_KEY, S3_KEY	Tutti	ELIMINATE	Sostituite da IAM Task Role.	
PROMPTS_DIR	agents	Bake-in Docker	Niente mount locali in Fargate.	

Tabella 17: Mappatura delle Variabili d'Ambiente.

9.2 Cifratura (AWS KMS)

Creare un'unica chiave **KMS Customer Managed Key** per l'MVP.

- **Utilizzatori per cifratura:** Servizio ElastiCache, Secrets Manager, S3. *MongoDB Atlas cifra a riposo con chiavi proprie, fuori da questa CMK.*
- **Utilizzatori per decifratura:** Task Execution Roles di backend e agents (per l'iniezione sicura all'avvio del container).

10 Budget e Osservabilità Infrastrutturale

10.1 Metriche e Allarmi Base

Tutti gli allarmi vanno instradati verso un unico Topic SNS (codeguardian-alerts), condiviso dal team di sviluppo via email.

Allarme / AWS Service	Metrica	Soglia di Attivazione
ECS Fargate	Task in esecuzione	< 1 (Desired Count) per > 5 minuti.
ECS Fargate	Utilizzo CPU / Memoria	> 80% per > 5 minuti.
ALB	Tasso di errore HTTP 5xx	> 5% su finestra di 5 minuti.
ElastiCache	Utilizzo CPU Cache	> 80% per > 5 minuti.

Allarme / AWS Service	Metrica	Soglia di Attivazione
Amazon Bedrock	Invocation Throttling	≥ 1 invocazione limitata in 5 minuti.

Tabella 18: Matrice degli allarmi infrastrutturali base.

Nota Implementativa

Le metriche del cluster **MongoDB Atlas** (CPU, connessioni, IOPS, replication lag) non sono esposte nativamente su CloudWatch: gli allarmi relativi si configurano con l'*alerting integrato di Atlas*, instradato verso lo stesso canale del team (email o, opzionalmente, integrazione Atlas → SNS/webhook). CloudWatch resta autoritativo per ECS, ALB, ElastiCache e Bedrock.

Correlazione dei Log tra Servizi

Ogni log strutturato emesso da **backend** e **agents** deve includere il campo **taskId** (già presente nel modello di dominio del PoC) in ogni riga, per l'intera durata dell'esecuzione di una Task. Questo consente di filtrare CloudWatch Logs Insights per una singola run end-to-end (avvio, tool-call verso GitHub, invocazione Bedrock, persistenza del Report) senza introdurre un sistema di tracing distribuito dedicato, non giustificato nei tempi dell'MVP.

10.2 Gestione Costi (AWS Budgets)

Azione Dev: Usare AWS Pricing Calculator per ottenere il Costo Mensile Stimato basato sui parametri di questo documento (1 NAT, 1 ElastiCache, 7 VPC Endpoint — incluso Atlas PrivateLink —, Compute ECS). **Il costo del cluster MongoDB Atlas M10 è fatturato separatamente da MongoDB (non compare in AWS Pricing Calculator) e va sommato alla stima; l'Interface Endpoint di PrivateLink è invece un costo AWS.**

- **Soglia 50%:** Informativa (Nessuna azione richiesta).
- **Soglia 80%:** Attenzione (Indagare eventuali loop di esecuzione).
- **Soglia 100%:** Superamento budget (Nessun blocco automatico configurato per evitare downtime bloccanti).

11 Tracciamento Requisito → Decisione AWS

Questa tabella mappa i requisiti funzionali, di vincolo e qualitativi (definiti nell'Analisi dei Requisiti) sulle scelte operative AWS implementate in questa configurazione. I requisiti puramente applicativi restano di competenza dello sviluppo software.

Requisito	Descrizione Sintetica	Decisione Infrastrutturale (AWS)	Riferimento
RV.15	Architettura e hosting basati su AWS.	Compute e servizi core su AWS: VPC, ECS (Fargate), ElastiCache, Bedrock, S3, CloudFront. Il database è MongoDB Atlas , servizio gestito di terze parti <i>deployato su AWS</i> in eu-south-1 e raggiunto via PrivateLink: hosting su AWS e data residency preservati, con una dipendenza da fornitore esterno (Parte II).	Tutto il doc.
RV.12	Database non relazionale basato su MongoDB.	Adozione di MongoDB Atlas (cluster M10, MongoDB nativo) in eu-south-1 , connesso via AWS PrivateLink.	§5
RV.13	Interfacciamento con LLM tramite AWS Bedrock.	Invocazione nativa Bedrock tramite Task Role IAM del container agents , via VPC Endpoint.	§6
RV.21	Schema Design della base dati non relazionale.	Demandato al team backend. Nessuno spike di compatibilità richiesto: Atlas è MongoDB nativo (driver Mongoose pienamente supportato).	§5
RV.4	Supporto a Repository privati, oltre ai pubblici.	Token GitHub cifrato in AWS Secrets Manager (RS.2), letto dal backend via Task Execution Role; l'accesso in HTTPS verso l'API GitHub avviene tramite il NAT Gateway, indipendentemente dalla visibilità del repository.	§3, §9
RV.14	Integrazione nativa con GitHub e GitHub Issues.	Nessun servizio AWS dedicato: il backend raggiunge le API GitHub in HTTPS in uscita (sg-backend → NAT Gateway), unico canale esterno oltre a Bedrock.	§3
RS.2	Token e credenziali memorizzati in formato cifrato.	Segreti instradati su AWS Secrets Manager, cifrati con KMS e iniettati nativamente da ECS al bootstrap.	§9
RS.3	Divieto di scrittura autonoma agenti su Git.	Il Security Group sg-agents blocca in uscita tutto il traffico Internet (0.0.0.0/0). Letture e scritture (apertura Pull Request) sono eseguite esclusivamente dal backend, mai dagli agenti.	§3, §13

Requisito	Descrizione Sintetica	Decisione Infrastrutturale (AWS)	Riferimento
RS.5	Non utilizzo codice per addestramento LLM terzi.	Transito tramite VPC Endpoint e invocazione via Bedrock, coperto dalla garanzia contrattuale AWS.	§6
RQ.4	Disaccoppiamento prompt dalla logica backend.	I prompt restano file esterni, ma il vincolo Fargate impone il bake-in nell'immagine Docker a ogni modifica.	§9
RQ.5	Accuratezza risposte agenti $\geq 85\%$.	Configurazione di modelli adeguati (Qwen3-Coder) e predisposizione di percorsi di scalata (Qwen3-235B).	§6
RQ.6	Tempo di risposta agente ≤ 5 minuti.	Adozione di ECS su Fargate al posto di AWS Lambda, incompatibile con lunghe esecuzioni o WebSocket.	§4
RV.6	Limitazione budget API.	Alert AWS Budgets su 3 soglie; limitazione della Policy IAM Bedrock alla sola famiglia Qwen3.	§10
RQ.8	Rate Limiting token API.	Rate limiter applicativo su Redis (token bucket per utente/IP) lato backend, a protezione delle chiamate verso Bedrock e GitHub; i limiti di account Bedrock restano una difesa di secondo livello, non il controllo primario.	§5

Tabella 19: Mappatura Requisiti - Decisioni Infrastrutturali AWS.

Requisiti Fuori Perimetro Infrastrutturale

I seguenti requisiti non compaiono in tabella poiché la loro soddisfazione dipende interamente dal codice sorgente o dalle policy di progetto, e non richiedono specifiche risorse o configurazioni AWS:

- **RV.1:** Natura deterministica dell'orchestratore.
- **RV.7:** Analisi supportata per TypeScript, JavaScript, Python.
- **RS.1:** Hashing password utente.
- **RS.4:** Validità logica delle credenziali Git esterne.
- **RV.16 – RV.20:** Vincoli di documentazione di progetto, Swagger, e versionamento codice.

Parte II

Motivazioni e Decisioni Architettureali

Questa sezione raccoglie le argomentazioni, i principi guida e le decisioni architetturali che hanno portato alla configurazione descritta nella Parte I.

12 Principi Guida e Vincoli di Progetto

Le scelte architetturali di questo MVP sono state guidate da un set di vincoli e principi applicati trasversalmente a tutti i livelli dell'infrastruttura. Invece di ribadirli per ogni singolo servizio, vengono dichiarati qui come fondamento delle decisioni successive.

- **Vincolo Tempo/Risorse (Il costo operativo):** Il team di sviluppo è composto da 6 persone con meno di due settimane per l'intera implementazione (infrastruttura inclusa). Questo ha imposto la scelta sistematica di servizi gestiti (es. Fargate al posto di EC2) e di topologie minimamente valide (es. singola istanza dati, singolo NAT) per abbattere il tempo di configurazione e manutenzione.
- **Principio di Migrazione ("Code-Freeze" infrastrutturale):** Nessuna sostituzione infrastrutturale deve toccare il codice applicativo applicativo o la logica di business. Le immagini Docker di `backend` e `agents` validate nel PoC restano le unità di deploy. L'adattamento avviene esclusivamente tramite iniezione di variabili d'ambiente e interfacce compatibili (es. MongoDB Atlas gestito al posto del container MongoDB del PoC, con lo stesso driver Mongoose).
- **Sicurezza Garantita per Costruzione vs Dichiarata:** Se un vincolo applicativo impone una restrizione (es. divieto di scrittura su GitHub per gli agenti), questo viene garantito a livello di rete (Security Groups, Routing) e non affidato unicamente alla correttezza del codice Python.
- **Infrastructure as Code (AWS CDK in TypeScript):** CDK è stato scelto rispetto a Terraform/HCL poiché il team possiede già una buona padronanza di TypeScript (React, NestJS). Inserire un nuovo linguaggio dichiarativo avrebbe impattato negativamente sul tempo di sviluppo.

13 Decisioni Architettureali (ADR) e Motivazioni

Di seguito vengono espanso e contestualizzate le decisioni architetturali cardine (ADR-AWS), raggruppate per dominio di competenza.

13.1 Identità, Accessi e Segreti

- **ADR-AWS-1 – Nessuna chiave statica a lungo termine:** L'autenticazione verso i servizi AWS (Bedrock, S3, ECR) avviene esclusivamente tramite **IAM Task**

Roles. È un miglioramento netto rispetto al PoC: le credenziali sono temporanee, auto-ruotanti e mai esposte nel codice. Si separano inoltre i ruoli di esecuzione (**Task Execution Role** per pull immagini/segreti) da quelli applicativi (**Task Role**).

- **ADR-AWS-5 – Singola Chiave KMS Condivisa:** Cifratura a riposo applicata a ElastiCache, S3 e Secrets Manager tramite un'unica chiave gestita dal cliente (MongoDB Atlas cifra a riposo con chiavi proprie, fuori da questa CMK). Un'architettura a chiavi isolate per servizio offrirebbe una limitazione del danno superiore, ma moltiplicherebbe la complessità delle policy IAM, compromesso non giustificato per un MVP di questa scala.
- **ADR-AWS-10 – Scrittura su GitHub: esclusivamente dal backend, mai dagli agenti:** La generazione automatica di Pull Request (RF.82, RF.83, RF.86) richiede un token GitHub con scope di scrittura (**repo**). Per non violare RS.3, l'apertura della PR non è mai eseguita dall'agente Python: l'agente produce solo una **Proposal** (diff) tramite la facade read-only esistente; è il backend NestJS, ricevuto l'output dell'agente, a invocare l'endpoint di scrittura delle API GitHub. L'isolamento di rete di **sg-agents** (zero egress) resta invariato: il confine di sicurezza non è lo scope del token ma il fatto che gli agenti non possiedono alcun percorso di rete verso l'esterno, nemmeno per la scrittura. La validazione umana della modifica proposta avviene interamente su GitHub (UC26.2.4/RF.63): il sistema non implementa un flusso di approvazione interno per questa operazione.

13.2 Rete, Sicurezza e Isolamento

- **ADR-AWS-2 e ADR-AWS-4 – Isolamento del livello dati e degli Agenti:** Redis (ElastiCache) risiede in subnet private senza IP pubblico; il database MongoDB Atlas è esterno alla VPC ma raggiunto esclusivamente via **AWS PrivateLink** (Interface Endpoint privato nelle subnet, nessun transito su Internet pubblico né dipendenza dal NAT). Il container **agents** non possiede alcuna route (nemmeno tramite NAT) verso 0.0.0.0/0. Questo rende impossibile per un agente contattare autonomamente GitHub o server esterni, garantendo il vincolo **RS.3** direttamente a livello di rete.
- **ADR-AWS-6 – Bedrock via PrivateLink:** Il modello LLM viene interrogato via VPC Endpoint Interface. Questo garantisce che il codice sorgente contenuto nei prompt non transiti mai su Internet pubblico, soddisfacendo la conformità sulla data residency e privacy.
- **ADR-AWS-7 – Debugging via ECS Exec:** L'accesso shell ai container per il debug avviene tramite AWS Systems Manager (SSM) Session Manager. Non vengono creati Bastion Host né utilizzate chiavi SSH. Questo elimina la vulnerabilità classica di istanze intermedie dimenticate aperte.
- **Scelta di Routing (Singolo NAT e 2 AZ):** Si adottano due Availability Zone poiché rappresentano il minimo tecnico richiesto dall'ALB. Si adotta un singolo NAT Gateway in **public-a** per contenere i costi continui. In caso di fail della AZ-A, il backend perderebbe temporaneamente l'accesso a GitHub. È un compromesso

calcolato e accettato per la scala dell'MVP.

13.3 Compute, Orchestrazione e Frontend

- **Perché Fargate e non Lambda/EC2:** EC2 è stato escluso per azzerare il tempo di amministrazione OS/host. Lambda è stato escluso poiché gli agenti hanno timeout lunghi (fino a 5 min) con stati conversazionali complessi (LangGraph), e il backend richiede connessioni WebSocket persistenti, pattern incompatibili col paradigma Lambda.
- **Desired Count pari a 1:** Mantenere 1 task per servizio semplifica drasticamente l'MVP. Scaling orizzontale del backend richiederebbe l'introduzione di sticky-sessions sull'ALB o un adapter Redis per la sincronizzazione dei WebSocket.
- **ADR-AWS-3 – Traffico interno via Cloud Map:** Il traffico tra backend e agenti avviene internamente tramite Service Discovery (namespace `.local`) protetto da HMAC. Questo evita di esporre API riservate del backend sull'Application Load Balancer pubblico.
- **ADR-AWS-8 e ADR-AWS-9 – ALB dietro CloudFront condiviso:** Il perimetro esclude l'acquisto di un dominio custom. L'ALB non può quindi ottenere un certificato SSL valido e ascolta in chiaro su porta 80. Viene posto dietro una distribuzione CloudFront (che fornisce TLS nativo gratis su dominio `*.cloudfront.net`) e l'ALB accetta traffico unicamente dai prefix-list di AWS Edge. Servendo sia S3 (Frontend) che ALB (Backend) sotto lo stesso dominio CloudFront, si rende il traffico *Same-Origin*, azzerando totalmente le configurazioni e i bug relativi alle policy CORS.

13.4 Scelte sul Livello Dati

- **MongoDB Atlas (gestito, esterno ad AWS):** Sostituisce il container MongoDB locale del PoC. Essendo MongoDB nativo, elimina il rischio di compatibilità sul driver e lo spike di validazione che DocumentDB avrebbe imposto. Il compromesso si sposta sulla natura del servizio: Atlas è un *data processor di terze parti*, mitigato deployandolo su AWS in `eu-south-1` e raggiungendolo via PrivateLink (traffico sul backbone privato AWS, data residency preservata). Resta una dipendenza da fornitore esterno con fatturazione separata.
- **Disponibilità del livello dati:** Il cluster Atlas M10 è un replica set a 3 nodi multi-AZ, quindi **HA per costruzione**: il database non è più un SPOF a istanza singola come con DocumentDB. Resta invece single-node **ElastiCache (Redis)**, senza repliche in questa fase; l'alta disponibilità della cache è esclusa dallo scope per motivi di budget e tempo.

13.5 Scelta dei Modelli LLM (Bedrock)

- **Esclusione di Llama 3.3 70B:** Il modello usato nel PoC non ha endpoint di inferenza nativi nella regione `eu-south-1` su Bedrock. Abilitarlo implicherebbe il

routing del codice sorgente analizzato verso regioni USA (cross-region inference), violando i principi di data residency e incrementando i costi operativi.

- **Adozione di Qwen3 (32B e 30B Coder):** Si adotta la famiglia Qwen3, nativa su Milano e in linea con il budget (\$0.15/\$0.60 per 1M token). L'agente Security sfrutta il modello **Qwen3-Coder-30B-A3B-Instruct** (MoE) che vanta un reliability score del 76.6% sul benchmark BFCL-v2 per il tool-calling, un requisito fondamentale per far sì che l'agente decida autonomamente quali file interrogare.
- **Fallback Plan:** Qualora il modello da 30B fallisse i test di accuratezza sul golden set (Piano di Qualifica), il team effettuerà la scalata su **Qwen3-235B-A22B**, sempre disponibile in **eu-south-1** per mantenere la conformità, con un modesto aumento dei costi.

14 Compromessi e Rischi Accettati

Il vincolo temporale per lo sviluppo dell'MVP ha reso necessario bilanciare la perfezione architetturale con il pragmatismo del rilascio. Invece di disseminare queste giustificazioni in ogni capitolo, vengono consolidate nella seguente tabella, che mappa i compromessi infrastrutturali accettati e il relativo Single Point of Failure (SPOF) o limite di sicurezza.

Risorsa / Scelta	Compromesso Architettuale	Rischio Accettato
NAT Gateway	Deployment di un singolo NAT Gateway allocato nella subnet public-a invece di uno per ogni Availability Zone.	Se la AZ eu-south-1a dovesse subire un disservizio, gli agenti perderebbero la connettività per leggere da GitHub. Lato frontend, l'errore è gestito come fallimento generico di orchestrazione (UC23), distinto da un errore di credenziale.
Ridondanza Compute	Subnet e ALB predisposti su 2 AZ, ma Desired Count: 1 per ogni servizio ECS: ogni servizio gira comunque su una sola istanza, in una sola AZ, alla volta. La topologia multi-AZ è quindi predisposizione di rete, non protezione attiva contro il fallimento del task.	In caso di crash del task o di disservizio sull'AZ ospitante, il servizio è irraggiungibile fino al riavvio automatico di ECS (nessun failover immediato su AZ alternativa). Rischio della stessa natura di quello già accettato per Auto-Scaling , qui esplicitato separatamente perché riguarda la distribuzione geografica, non solo il numero di repliche.

Risorsa / Scelta	Compromesso Architetture	Rischio Accettato
ElastiCache (Redis)	Nodo singolo (nessuna replica, no modalità cluster). Il DB su Atlas M10 è invece multi-AZ e non è più un SPOF.	SPOF residuo limitato a cache e coda BullMQ. Cluster ElastiCache Multi-AZ non giustificato per un carico da validazione MVP.
MongoDB Atlas (fornitore esterno)	Database gestito da terze parti, fuori dall'account AWS, con fatturazione separata e piano di controllo non gestito via CDK.	Dipendenza da un provider esterno e da PrivateLink per la connettività. Data residency preservata (deploy su AWS eu-south-1); un disservizio di Atlas o del Private Endpoint interrompe il livello dati.
AWS KMS	Utilizzo di un'unica chiave crittografica condivisa per ElastiCache, S3 e Secrets Manager.	Una compromissione della chiave esporrebbe questi livelli di persistenza AWS (MongoDB Atlas cifra a riposo separatamente). Si evitano complessità e moltiplicazione delle policy IAM.
Application Load Balancer	ALB senza certificato TLS pubblico e senza dominio custom, in ascolto solo su HTTP/80 (dietro CloudFront). Conseguenza diretta dell'esclusione del Dominio Custom (Route53) dal perimetro MVP (§2.2): senza un dominio proprio non è possibile emettere un certificato ACM valido per l'ALB.	Il tratto di rete tra l'edge CloudFront e l'ALB non è cifrato (traffico comunque ristretto alla rete privata AWS, non instradato su Internet pubblico). Il rischio è accettato in coerenza con la decisione di non acquistare un dominio per l'MVP (ADR-AWS-8); introdurlo richiederebbe di riaprire quella decisione, non solo di configurare un certificato.
Auto-Scaling	Desired count fisso a 1 per i servizi ECS. Nessuna policy di autoscaling configurata.	Irraggiungibilità momentanea in caso di task failure durante il tempo di riavvio (Drop SLA). La resilienza (retry, risincronizzazione post-riconnessione) è responsabilità applicativa backend/frontend, da definire nell'MVP riusando l'approccio del PoC.

Risorsa / Scelta	Compromesso Architettuale	Rischio Accettato
AWS WAF	Nessun Web Application Firewall configurato a livello Edge (Cloud-Front/ALB).	Nessuna protezione infrastrutturale contro DDoS Layer 7 o abusi di throttling applicativo da client anomali.
Token GitHub	Token unico con scope di scrittura (repo), condiviso tra letture e apertura PR. Nessuna segmentazione read/write a livello di credenziale.	Una compromissione del backend (unico detentore del token decifrato) consentirebbe scritture sul repository. Il rischio è mitigato dall'isolamento di rete degli agenti e dalla cifratura KMS del token in Secrets Manager, ma non eliminato: non c'è un secondo token a permessi ridotti per le sole letture.
Backend — Sizing task per PDF	Task backend dimensionato per un servizio REST leggero (0.5 vCPU / 1 GB, Tabella 7), esteso senza revisione dedicata a generare e streammare PDF nello stesso container.	Non verificato dal team se il carico di generazione PDF (specialmente in concorrenza con task WebSocket attivi) rientri nel budget di memoria. Rischio contenuto se la generazione usa una libreria non basata su browser headless; da confermare con un test di carico mirato prima del rilascio, non con un ridimensionamento preventivo.

Tabella 20: Matrice dei Compromessi e Rischi Accettati per l'MVP.

15 Limiti di Piattaforma (Fargate e Prompt)

Il PoC consentiva la modifica in tempo reale dei prompt passati ai modelli LLM grazie al montaggio di volumi (bind mount) da file system locale su Docker Compose. **AWS Fargate non supporta i bind mount da directory locali.** Per evitare l'introduzione di un file system condiviso (Amazon EFS), che comporterebbe costi continui e overhead di configurazione ingiustificati per l'MVP, **i file dei prompt devono essere inclusi staticamente (bake-in) all'interno dell'immagine Docker.**

Conseguenza operativa: Il disaccoppiamento logico richiesto da **RQ.4** resta soddisfatto (i prompt sono file YAML isolati dal codice Python), ma ogni modifica al testo di un prompt richiederà ora la ricompilazione dell'immagine

e un nuovo deploy del servizio `agents`.

Il tuning effettivo dei prompt avviene comunque in locale via Docker Compose (bind mount, come nel PoC): il bake-in impatta solo la promozione di una versione già validata verso l'ambiente Fargate, non il ciclo di iterazione. Un'alternativa a costo contenuto (prompt su S3, scaricati a runtime) resta valutabile in una fase successiva, se il tuning dovesse richiedere iterazione diretta contro Fargate/Bedrock.